Creating a vector store: The document chunks need to be converted into embeddings to be stored in a vector store. We use one of Google's text embeddings model "text-embedding-005" as the vector embedding engine.

```
embeddings_model = VertexAIEmbeddings(model_name="text-embedding-005")
```

The latest version of Redis has introduced support for vector search and is used as the vector database for our application. Populating Redis involves multiple steps – creating the Redis client, initialising the index for vector search, and creating the vector database instance by passing the required details.

```
client = redis.Redis.from_url(REDIS_URL)
index_config = HNSWConfig(
    name=»rag_demo", distance_strategy=DistanceStrategy.COSINE, vector_size=768)
RedisVectorStore.init_index(client=client, index_config=index_config)
For creating a vector store, we use the RedisVectorStore class from langchain_google_memorystore_redis.
redis_vector_db = RedisVectorStore(
    index_name="rag_demo",
    embeddings=embeddings_model,
    client=client)
```

Storing document embeddings in Redis: The final step in data preparation is adding the document chunks to the vector database. When the chunks are being added, for each chunk, Redis will create embeddings using the specified embedding model. The generated embeddings are stored in the index and ready for querying using vector search.

```
redis_vector_db.add_documents(doc_chunks)
```

Data serving layer

Once the data is prepared and made available in the vector store, the next step is to consume it in the data serving layer for providing responses to the user's queries.

Creating the retriever: From the Redis vector database, a retriever is created to perform similarity search within the vector store and return the top-k document chunks that have the lowest vectoral distance with the incoming user query.

```
retriever = redis_vector_db.as_retriever(search_type="similarity",search_kwargs={"k": 3})
```

Querying the LLM: The next step is to initialise the actual LLM that will do the final processing to generate the response for the user. In this application, we use Google's gemini-2.0-flash model and initialise the LLM using VertexAI class from langchain_google_vertexai.

```
llm = VertexAI(project=PROJECT_ID, location=LOCATION,
    model_name="gemini-2.0-flash",
    max_output_tokens=1000, temperature=0.05, top_p=0.8, top_k=40, verbose=True )
```

The final step is to create a query engine. For this, we use *RetrievalQA* chain from LangChain.

```
qa = RetrievalQA.from_chain_type( llm=llm, chain_type="stuff", retriever=retriever )
```

This QA chain can now be used to answer questions from users by retrieving context from the private enterprise data chunks that are stored in the vector database.

```
query = "What is the revenue of Alphabet in 2024? What is the increase compared to 2023"
results = qa({"query": query})
print("Query:", query)
print("Response:\n", results['result'])

Query: What is the revenue of Alphabet in 2024? What is the increase compared to 2023

Response:
 The revenue of Alphabet in 2024 was $350.018 billion. The increase compared to 2023 was $42.624 billion.
```

Providing a UI front-end using Gradio: The Gradio framework has gained popularity due to its ability to support complex UI functions with a few lines of code. For our current application, a simple front end can be provided using Gradio. Users can query against documents and obtain answers from the LLM with information that references the enterprise documents that were provided via the data sources.

The complete Python notebook for this sample application is available at *https://github.com/lokeshchenta/genai-google-gcp/tree/main/rag-chatbot.*

Implementing an RAG-based genAI chatbot for private enterprise data has become simpler with frameworks like LangChain and supporting platforms like Vertex AI. Cloud platforms like Google have even started providing managed services like Dialogflow and Vertex AI Agent Builder. These provide low-code, no-code mechanisms for simple RAG applications. This has further simplified the efforts needed to build an enterprise chatbot. Development teams now have a variety of options to choose from, including no-code solutions and custom implementations. The decision will depend on several factors such as the complexity of